Web design and Development Assignment (CS323)

Department of computer science

Batch DRB2302

Section A

Group members

1. Bilal Mohammed
2. Biruke Melaku
3. Kaleb Astatke
4. Michael Lemma
5. Yosef Seyoum

Submitted to: Mr.Addis Hailu

Submission Date: Saturday-January-31-2026

Acknowledgment

We would like to express our sincere gratitude to our teacher, Mr. Addis, for giving us this assignment. Your passion for teaching and your dedication to helping us understand concepts and frameworks have truly motivated us to explore the subject more deeply. We greatly appreciate the knowledge and skills you have shared with us, which have enhanced our learning experience and helped us gain a clearer understanding of the field. Thank you for being an inspiring teacher and a positive influence on our academic journey.

- Group members -

Project Documentation: Campus Connect

Problem Statement

In the modern academic environment, students are frequently confined to interactions within their own university campuses, classes, or batches. This limitation restricts opportunities for cross-campus collaboration, knowledge sharing, and networking among peers studying similar subjects or pursuing related interests. As a result, students miss out on diverse perspectives, shared resources, and potential team formations that could enhance their educational and professional growth. Campus Connect addresses this by providing a unified platform to connect students from different universities, enabling seamless communication, event organization, and community building.

Objectives

- The main goal of Campus Connect is to create an inclusive, cross-university networking platform that promotes collaboration and expands students' social and academic networks.
- To enable cross-campus connections: Allow students to discover and connect with peers from other universities based on shared interests, skills, and academic fields.
- To facilitate communication and knowledge exchange: Provide tools for messaging, group creation, and discussion to share experiences, resources, and ideas.
- To support event management: Enable users to create, promote, and participate in events that transcend individual campuses.

Functional and Non-Functional Requirements

Functional Requirements (What the system does)

- User Registration and Authentication: Students register with university details, upload ID for verification, and log in securely. Admins approve/reject registrations.
- Profile Management: Users can update profiles with bio, skills, interests, social links, and photos.
- Messaging System: One-on-one chats, group messaging, with support for text and file uploads.
- Group Management: Create, join, leave, or moderate groups tied to universities.
- Event Management: Create events, view attendance, with visibility controls
- File uploading feature
- Notifications: Real-time alerts for messages.
- Admin Controls: University admins manage approvals, bans, and block; super admins oversee everything.

Non-Functional Requirements (How the system performs)

- Scalability: Handle multiple universities and high user volumes using MongoDB's flexible schema and indexing.
- Security: Use hashed passwords, JWT for API authentication, secure file storage

- Performance: Optimize queries with indexes (e.g., on timestamps for messages/posts) to ensure fast loading of feeds and chats.
- Usability: Responsive UI, with intuitive navigation.
- Privacy: Support profile visibility settings (public, private) and data deletion options.

System Architecture (MERN Stack Overview)

Campus Connect is developed using the MERN stack, which provides a full-stack JavaScript solution for efficient development and deployment.

- MongoDB (Database): A NoSQL database that stores unstructured data in collections like users, groups, and messages. It supports complex queries and relationships via ObjectIDs, with built-in validation for data integrity.
- Express.js (Backend): Handles server-side logic, API routing, authentication middleware JWT, and integrations like file uploads using Multer.
- React (Frontend): Builds dynamic, component-based UIs for dashboards, chats, and forms, managing state with Context API for real-time updates.
- Node.js (Runtime): Powers the server, enabling asynchronous operations, REST API handling via Express.js, MongoDB/Mongoose database connections, JWT authentication, file uploads with Multer, and static asset serving for profiles and message attachments.

The architecture follows a client-server model: React frontend communicates with Express APIs, which interact with MongoDB. Security is enforced via middleware(helmet)

System Features

- Verification Workflow: Students upload IDs; status stays "pending" until admin approval, ensuring authenticity.
- Real-Time Messaging: Supports chats and group messages with file attachments, unread counts, and read receipts.
- Group Communities: Users create/manage groups for topics/projects, with roles (member, moderator, admin) and privacy settings.
- Event Organization: Create events with details like date, location, and images; users and view attendance.
- Profile Customization: Add bio, skills, interests, and social links to showcase personal branding.
- Profile Visibility: Control who can view profile (public, university, connections, private)
- File Uploads: Profile pictures, cover photos, ID verification documents, and message attachments

ER Diagram / Data Model (Optional)

The data model is relational in nature but implemented in MongoDB's document-oriented style. Key entities and relationships:

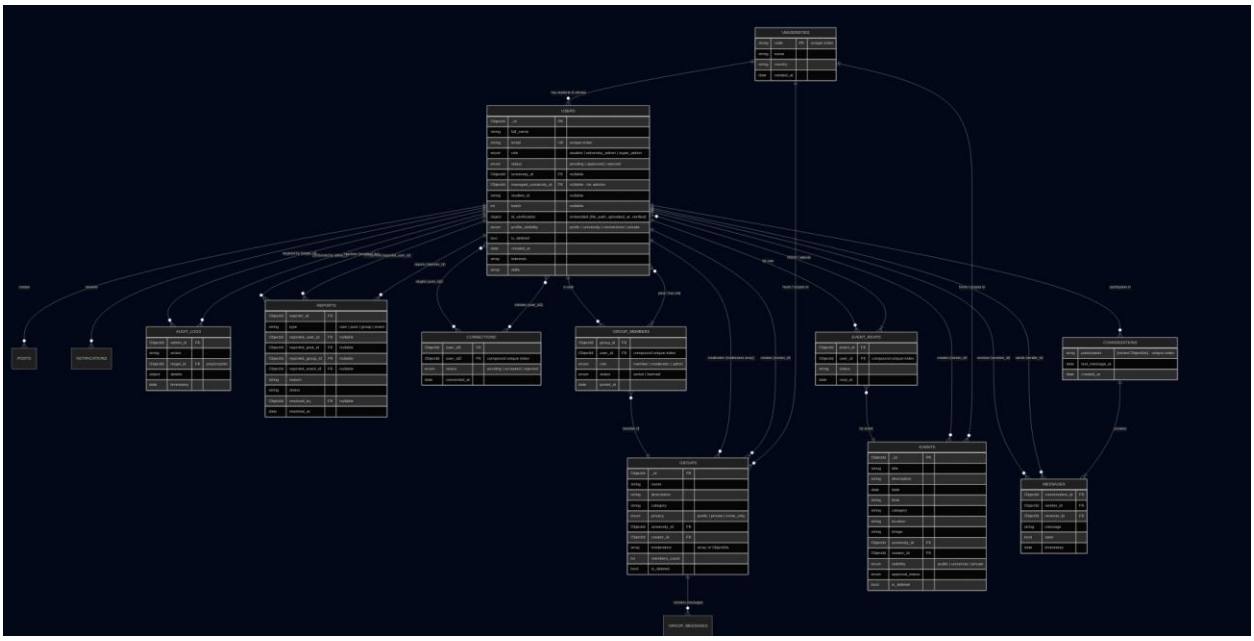
- Users (central entity): Linked to Universities (via university_id), Groups (via Group_Members), Events, Messages (as sender/receiver), Notifications (as recipient).
- Groups: Tied to Universities and Users (creator/moderators/members); contains Group_Messages.
- Events: Tied to Universities and Users (creator/attendees); has RSVPs.
- Conversations/Messages: Conversations link two Users; Messages reference Conversations.
- Universities: Parent entity for Users, Groups, and Events.

Relationships:

One-to-Many: User to Posts/Messages/Events/Groups (as creator).

Many-to-Many: Users to Groups (via Group_Members), Users to Events (via Event_RSVPs), Users to Users (via Connections).

One-to-One: User to ID_Verification (embedded object).



API Endpoints (Optional but Recommended)

The APIs are RESTful, secured with JWT authentication (auth middleware), and handle operations like file uploads.

- User Management (/api/users):GET /test, GET /me, PATCH /me, DELETE /me, GET /search, POST /me/profile-picture
- Groups (/api/groups):GET /test, GET / (list), GET /mine, POST /, DELETE /:id, POST /:id/join, POST /:id/leave, GET /:id/messages, POST /:id/messages

Additional implemented: PUT /:id/privacy, POST /:id/add-member, DELETE /:id/remove-member/:memberId

- Direct Messaging (/api/messages):GET /test, GET /conversations, GET /thread/:otherUserId, POST /send, GET /unread-count, POST /thread/:otherUserId/read, POST /contact
- Reports (/api/reports):Only GET /test is implemented ⚠ (no report submission/moderation endpoints)

- Security: All endpoints (except /test and /contact) use JWT authentication via auth.middleware.js.

- File Uploads

  Supported on:

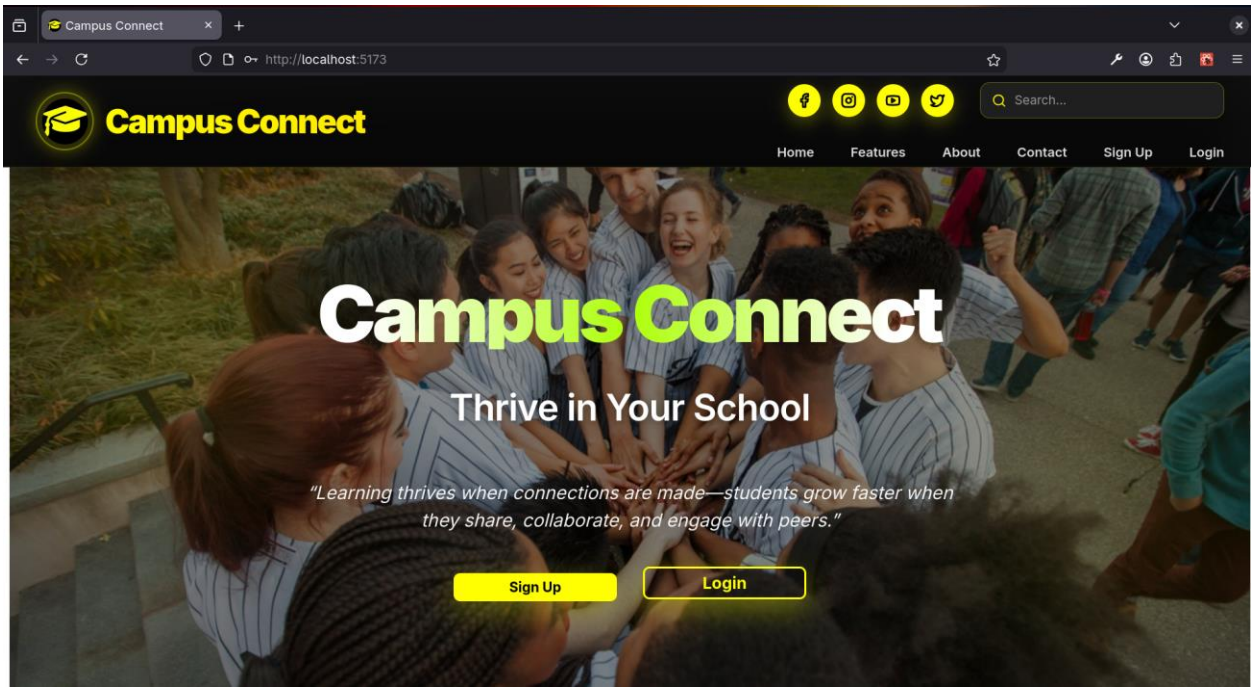
  POST /api/users/me/profile-picture (field: profile_picture)

  POST /api/groups/:id/messages (field: file)

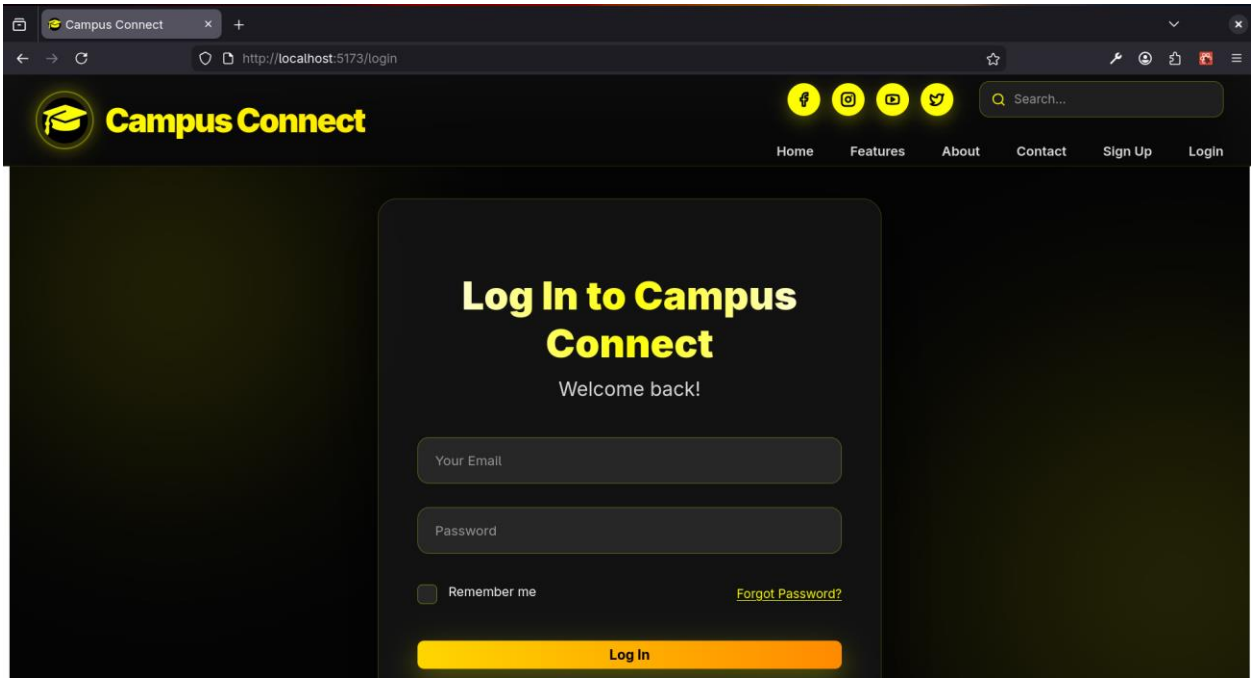
  POST /api/messages/send (field: file)

Screenshots

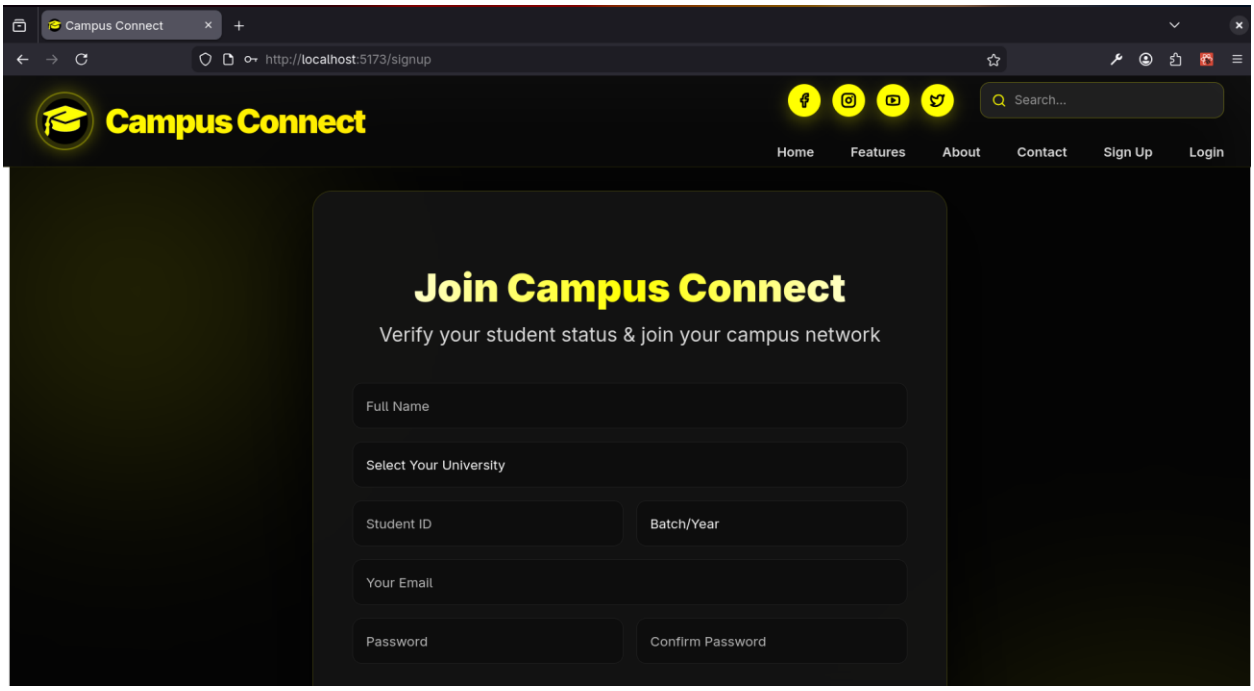
Homepage:



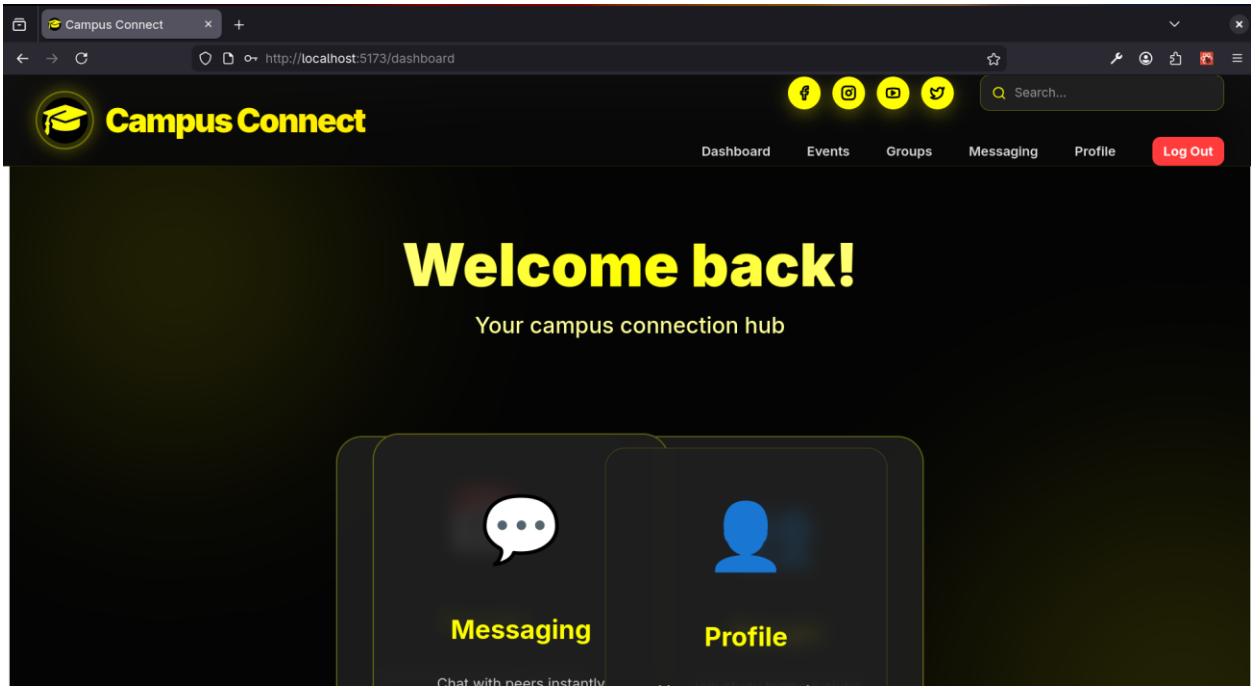
Login:



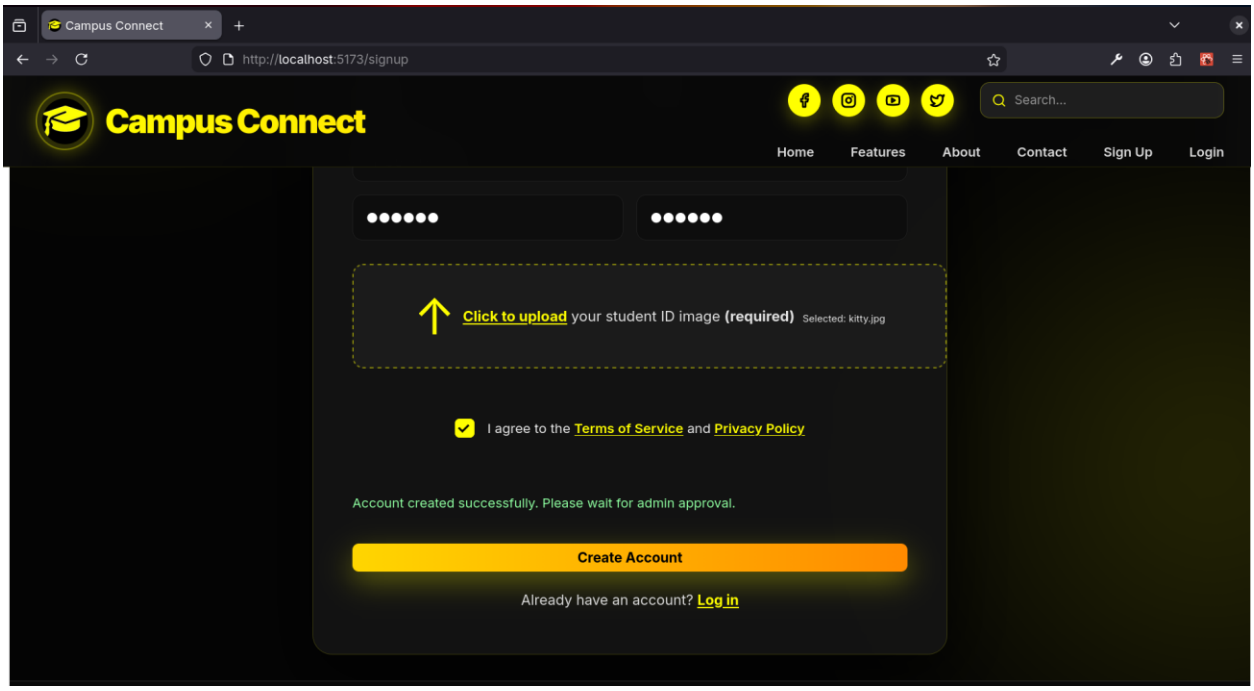
Signup:



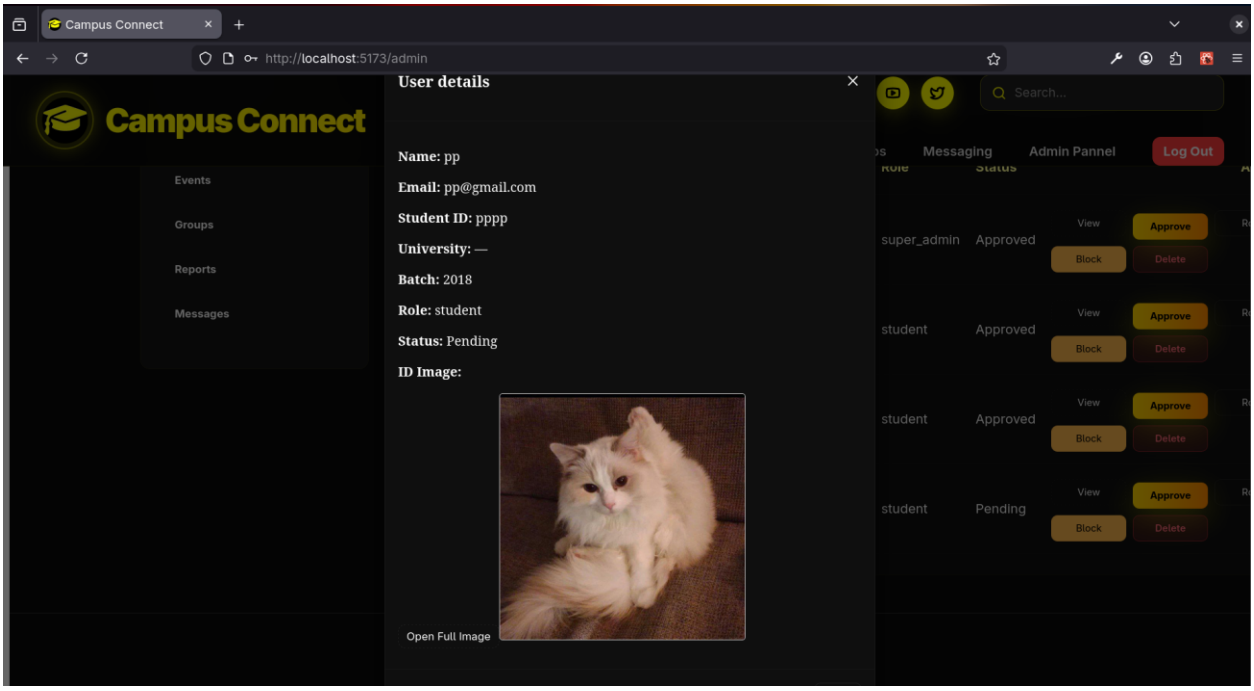
Dashboard:



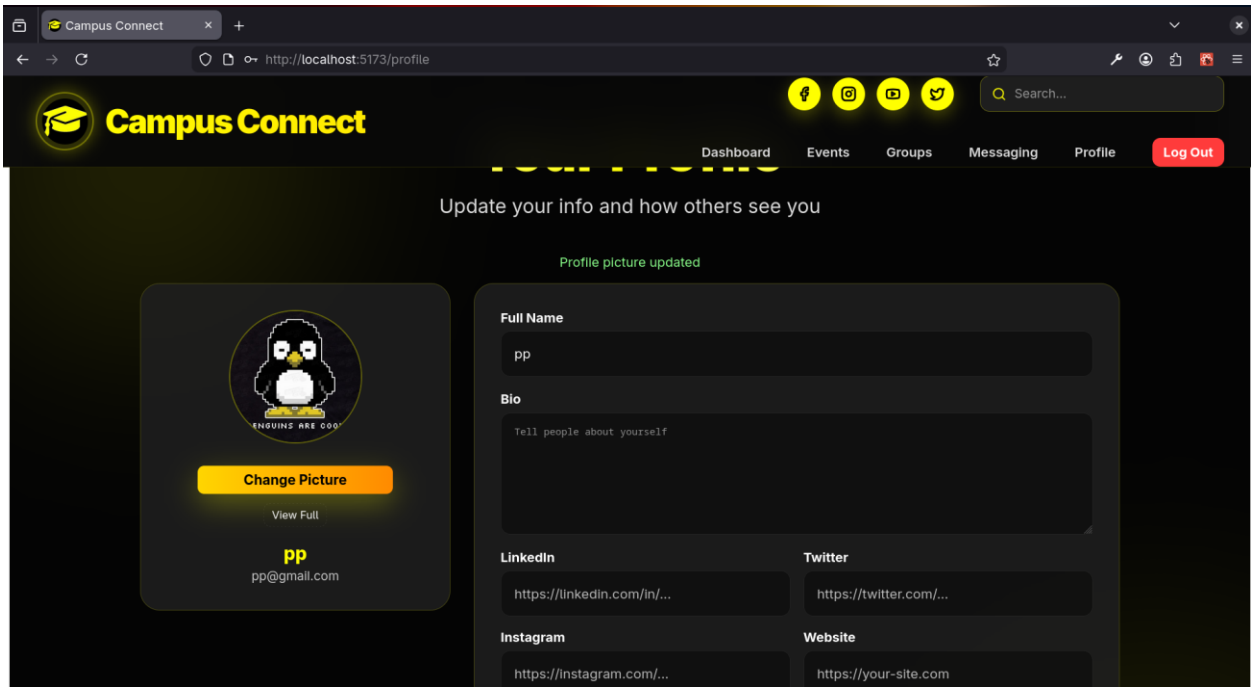
Create:



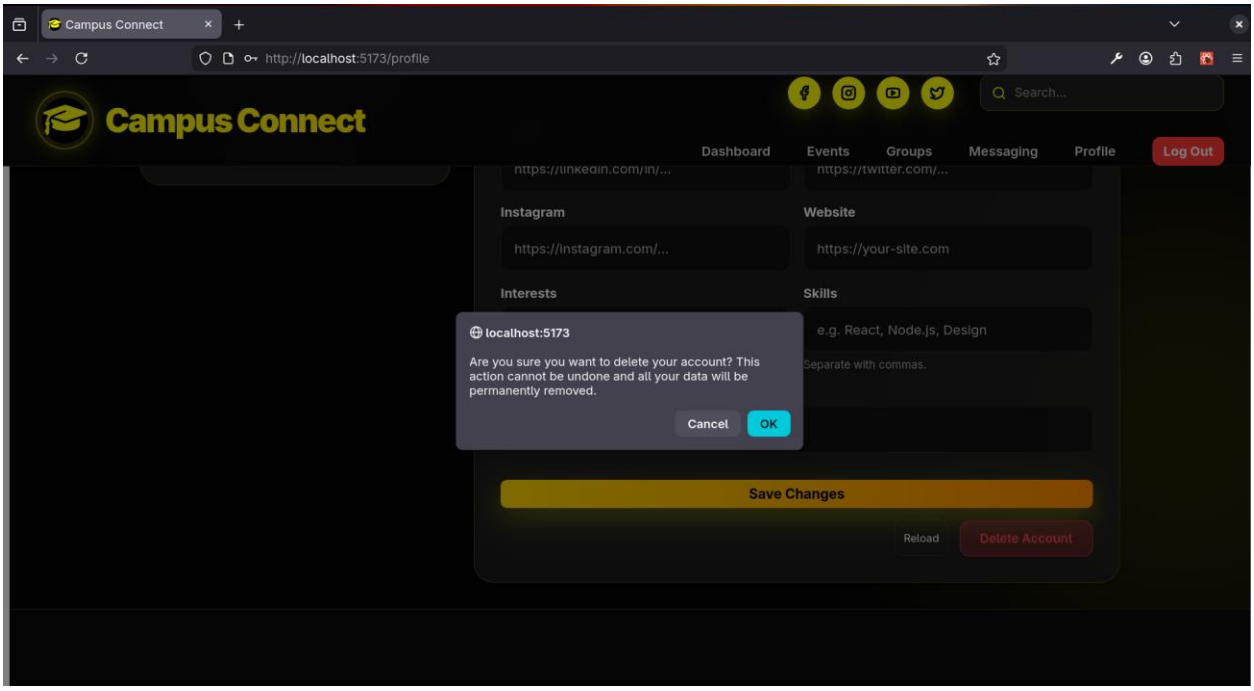
Read:


Campus Connect
http://localhost:5173/admin
Campus Connect
Events
Groups
Reports
Messages
User details
Name: pp
Email: pp@gmail.com
Student ID: pppp
University: —
Batch: 2018
Role: student
Status: Pending
ID Image:
Open Full Image
Search...
Messaging
Admin Pannel
Log Out
super_admin
Approved
student
Approved
student
Approved
student
Pending
View
Approve
Block
Delete

Update:


Campus Connect
http://localhost:5173/profile
Campus Connect
Search...
Dashboard
Events
Groups
Messaging
Profile
Log Out
Update your info and how others see you
Profile picture updated
Change Picture
View Full
pp
pp@gmail.com
Full Name
pp
Bio
Tell people about yourself
LinkedIn
https://linkedin.com/in/...
Twitter
https://twitter.com/...
Instagram
https://instagram.com/...
Website
https://your-site.com

Delete:



Conclusion & Recommendations

Conclusion

Campus Connect effectively resolves the issue of isolated student networks by offering a secure, MERN-based platform for cross-campus interaction. Through features like messaging, groups, and events, it promotes knowledge sharing and collaboration, fostering a vibrant community that enhances academic and social experiences.

Recommendations

- Mobile Expansion: Develop a mobile app using React Native for push notifications and better accessibility.
- AI Enhancements: Integrate machine learning for personalized recommendations of connections or groups based on interests/skills.

- Advanced Analytics: Add dashboards for admins to track engagement metrics.
- Integration with External Services: Connect to calendar apps (e.g., Google Calendar) for event syncing or OAuth for university logins.
- Direct School Integration: Expand partnerships with institutions to automate account creation, ensuring every enrolled student has immediate access to Campus Connect without administrative delays.